

Kimi K3 技术分享

开源前沿智能 · 2.8T MoE / 1M 上下文 / 原生多模态

重点：结构设计背后的「为什么」 · 预训练 · 预训练 Infra

（结构基础已由前序分享覆盖，本场只讲动机与原理；强化学习简化）

本场路线图（约 1 小时）

五个部分，Infra 篇幅最重

- **Part 0 · 定位与总纲（约 5 min）：** 为什么做 3T · 三轴信息流
- **Part 1 · 结构：为什么这么设计（约 18 min）**
 - KDA / Gated MLA · NoPE · Attention Residuals · Stable LatentMoE · Per-Head Muon · 对比 DeepSeek/Qwen
- **Part 2 · 预训练（约 10 min）：** Scaling Law · 长上下文 · QAT
- **Part 3 · 预训练 Infra（重点，约 20 min）**
 - 并行组合 · KDA 协同 (FlashKDA/KCP) · MoonEP 完美负载均衡 · 显存组合拳 · 多模态 encoder
- **Part 4 · RL（简化，约 5 min）+ 收尾**
- **三个记忆钩子：KDA 管『跨 token』，AttnRes 管『跨 layer』，LatentMoE 管『跨 channel』**

要点：今天不重复结构基础，重点是『为什么这么设计』与预训练 Infra。



Part 0

定位与总纲

为什么是 3T ? 三轴信息流

为什么把底座推到 3T 级？

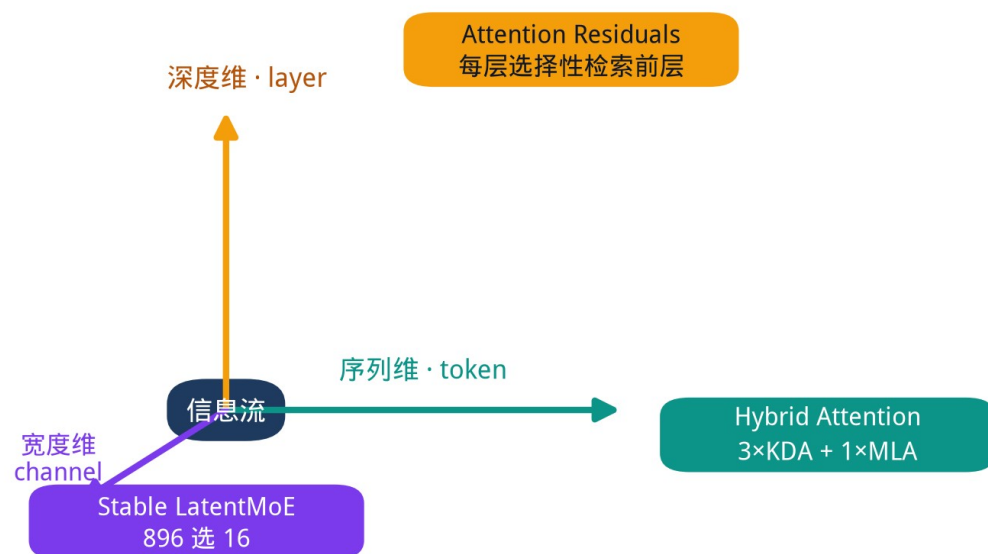
两条 scaling 轴，K3 选择同时推进

- LLM 的 scaling 有两条轴
 - 轴一 · 预训练规模：更大模型 + 更多数据
 - 轴二 · test-time compute：RL + 推理 effort + 长程交互
- 现状：开源在『轴二』猛进，但『轴一』停在 1T 级附近
 - 风险：都在相似规模底座上堆 RL → 开源互相收敛，与闭源差距扩大
- K3 的选择：两轴同时推到前沿
 - 底座到 3T 级，同时把 RL / effort / 交互推到 1M 上下文
- 结果：2.8T 总参 / 104B 激活 / 1M 上下文，整体 scaling 效率相比 K2 约 2.5×

要点：别人卷 test-time，K3 选择『底座 + test-time』两条轴一起卷到前沿。

总纲：沿三个维度『扩展信息流』

所有结构创新都在回答同一个问题



- 核心问题：信息如何更高效地流动？
- 序列维（token）：Hybrid Attention = 3×KDA + 1×MLA → 长序列高效混合 + 全局兜底
- 深度维（layer）：Attention Residuals → 每层选择性检索所有前层
- 宽度维（channel）：Stable LatentMoE 896 选 16 → 稀疏扩展通道容量
- 训练稳定：Per-Head Muon + QB + 有界化 → 在 2.8T/896 专家 /MXFP4 下稳住优化
- 记忆钩子：KDA 跨 token · AttnRes 跨 layer · LatentMoE 跨 channel

要点：记住这三条轴，后面每个模块都挂在它上面。

K2 → K3 规格对照

改了什么，一目了然

- 总参 / 激活: 1.04T/32.6B → 2.78T/104B (+167% / +220%)
- 层数: 61 → 93 ; 注意力: 全 MLA → 69 KDA + 24 Gated MLA (3:1)
- 路由专家 / 激活: 384/8 → 896/16 (+2 shared)
 - Latent MoE 维: 无 → 3584 (0.5× hidden)
- 激活函数: SwiGLU → SiTU-GLU ; 位置编码: RoPE → NoPE
- 训练上下文: 128K → 1M (8×)
- 优化器: Muon → Per-Head Muon
- 量化: 无 → MXFP4 权重 / MXFP8 激活 (QAT)
 - 视觉: 新增 MoonViT-V2 (401M , 27 层, 从零训练)

要点: 几乎每个模块都动了刀——这也是为什么需要一整套新 Infra。



Part 1

模型结构：为什么这么设计

基础从简，重点讲动机与原理

序列维：Hybrid Attention = 3×KDA + 1×MLA

为什么是混合，而不是纯线性

- 每个 block：3 层 KDA（线性）+ 1 层 Gated MLA（全局），末尾再补 1 层 MLA
- 为什么混合？
 - KDA 承担绝大多数 token mixing，递归状态固定大小 → 长序列便宜
 - 线性注意力的软肋 = 长程精确召回 → 用 1/4 全局 MLA 兜底
 - 3:1 是『效率 vs 召回』的工程折中（继承自 Kimi Linear）
- 直接收益：1M 上下文下 KV cache 与算力显著下降

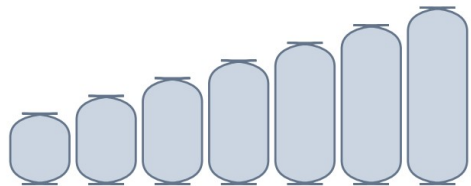


要点：DeepSeek 用全 MLA、Qwen 用 GQA；K3 用『线性为主 + 全局兜底』换长上下文效率。

KDA 是什么：带通道级遗忘门的 delta rule

用固定大小 RNN 记忆做『可控遗忘 + 精确改写』

Softmax 注意力：KV cache 随长度增长



内存 $O(T) \uparrow \uparrow$

KDA：固定大小递归状态 S



内存 $O(1)$ · 易传输/复用

delta-rule 更新：先「按 key 方向擦除旧关联」，再「写入新 kv^T 」

通道级遗忘门 $\text{Diag}(\alpha)$ ：每个特征维独立控制记忆衰减

（比 Gated DeltaNet / Mamba-2 的「头级标量门」更细粒度）

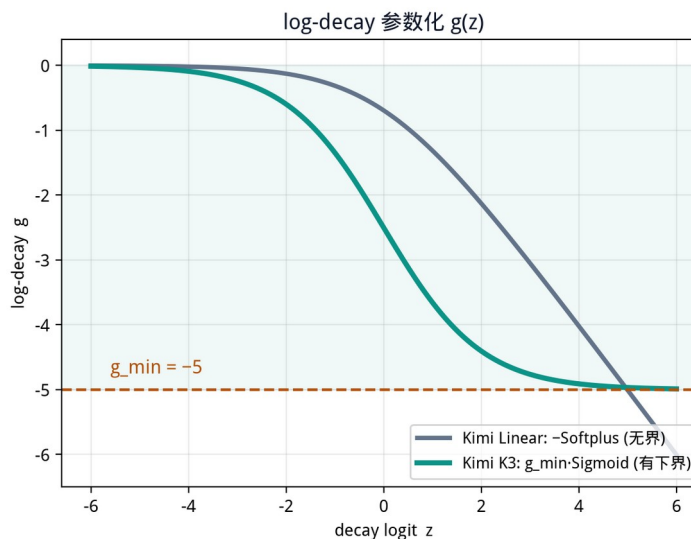
- 状态 S 是固定大小的递归记忆（不像 KV cache 随长度增长）
- delta rule：先『按 key 方向擦除旧关联』，再『写入新 $k \cdot v$ 』
- 通道级遗忘门 $\text{Diag}(\alpha)$ ：每个特征维独立控制记忆衰减
 - 比 Gated DeltaNet / Mamba-2 的『头级标量门』更细粒度
 - $q/k/v$ 走 ShortConv + Swish， q/k 再 L2Norm（细节从略）

要点：固定大小状态 = 长序列便宜、易传输 / 复用（这点后面 Infra 会反复用到）。

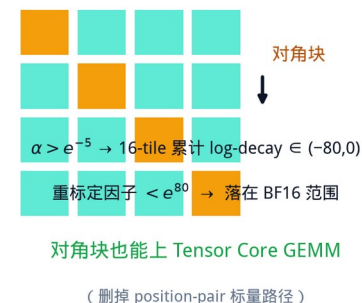
KDA 演进①：给衰减门『加下界』→ 系统大收益

算法 - 系统协同的典型缩影

- 问题：chunkwise 里要用累计衰减的倒数重标定 key，倒数会无界增长、BF16 溢出
 - Kimi Linear 的对角块只能逐位置对计算 → 上不了 Tensor Core，成瓶颈
- K3 改法：把 log-decay 从『无界负 Softplus』换成『带下界的缩放 sigmoid』
 - 数学后果：保留因子有下界 → 重标定因子落在 BF16 范围
- 系统收益：对角块也能用稠密 Tensor Core GEMM，删掉标量路径



系统后果



要点：一个纯数学的『下界』，换来 kernel 的硬件友好——这类协同全场会反复出现。

KDA 演进② + Gated MLA + NoPE

全秩输出门 · 全局层为什么不加位置编码

- **KDA 演进②：输出门从『低秩』换成『输入相关的全秩』投影**
 - 让每个 token 更充分地调制从递归记忆读出的通道 → 表达力更强
- **Gated MLA：把 KV 压成低维 latent 缓存，保留全局注意力（源自 DeepSeek-V2）**
- **NoPE：所有 MLA 层不加任何显式位置编码**
 - 位置信息由中间的 KDA 层通过递归门控隐式提供
 - 副产品：扩上下文零改动——不用重调 RoPE、不用 YaRN → 直接外推 1M

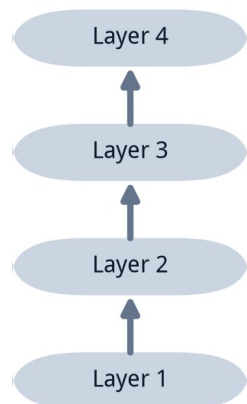
要点：NoPE 是 K3 能『直接外推到 1M』的关键之一。

深度维：为什么需要 Attention Residuals

标准残差 = 深度维的 RNN

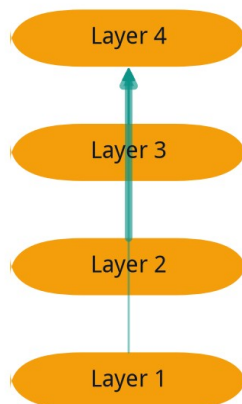
标准残差 = 深度维 RNN

固定单位权重逐层累加
早层信息被稀释、幅度 $O(L)$ 增长



Attention Residuals = 深度维 softmax

每层用可学 pseudo-query
对所有前层做 softmax 选择



α

- 洞见：PreNorm 残差 = 每层用固定单位权重把所有前层加起来 = 深度维 RNN

- 三个后果

- 无选择性访问：attention 层和 MLP 层拿到同一个聚合态
- 不可逆损失：早层信息被埋，深层无法选择性找回
- 输出膨胀：hidden 幅度随深度 $O(L)$ 增长，稀释每层贡献、破坏稳定

- 类比：序列维当年用 attention 取代 RNN 递归——深度维也该如此

要点：把『序列维的 attention 革命』搬到深度维。

Attention Residuals + Block 版

深度维 softmax + 把开销降到 $O(Nd)$

- **Full AttnRes** : 每层用可学 pseudo-query , 对『所有前层』做 softmax 选择

- 理论: 标准残差 = 深度维线性注意力; AttnRes = 深度维 softmax 注意力

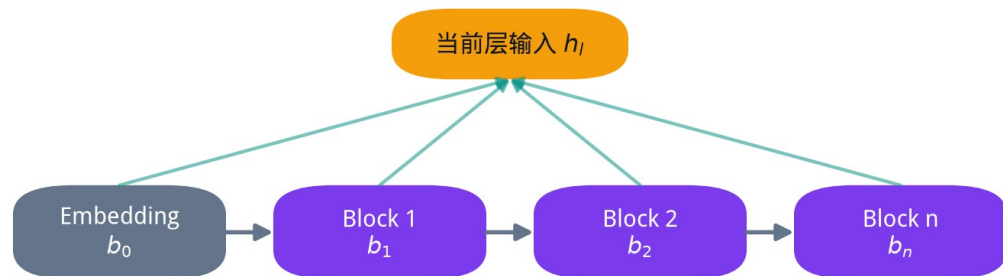
- **Block AttnRes** : L 层切成 N 个 block , 块内标准残差、块间才做注意力

- 显存 / 通信 $O(Ld) \rightarrow O(Nd)$; K3 用 $N \approx 8$ (93 层切 8 块)

- 收益: 同算力 $\approx$ baseline 的 $1.25\times$ · hidden 幅度有界 · 梯度更均匀

- 推理友好: block 间可并行 + online softmax 严格等价 $\rightarrow$ 延迟开销 $<2\%$

Block AttnRes: L 层 $\rightarrow$ N 个 block, 块间做全注意力 (K3: $N \approx 8$)



块内标准残差求和 $\rightarrow$ 块间仅对 N 个块表示做注意力

显存/通信 $O(Ld) \rightarrow O(Nd)$

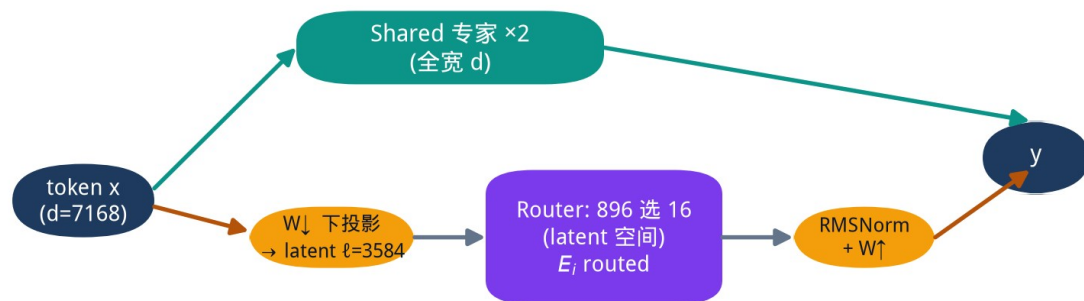
同算力 $\approx$ baseline 的 $1.25\times$ · 推理开销 $<2\%$

要点: $N \approx 8$ 就能拿回大部分收益, 且推理几乎不额外花钱。

宽度维：Stable LatentMoE 让『896 选 16』可负担

分离模型宽度与路由专家宽度

Stable LatentMoE：分离「模型宽度」与「路由专家宽度」



只在压缩 latent 空间里路由 → 896 选 16 的通信/显存才付得起 (稀疏度 ≈ 56)

RMSNorm 治「4 连乘病态」的激活爆炸 · SiTU-GLU 有界 · QB 均衡

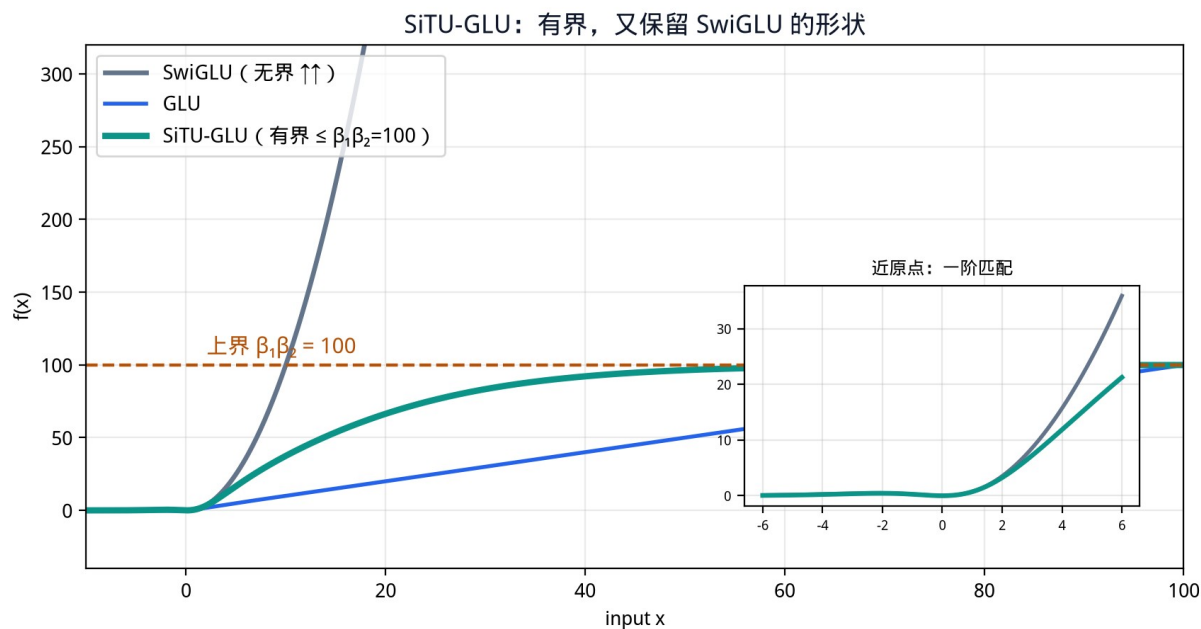
- 常规 MoE：每个被选专家吃完整 d 维 token → 通信 / 权重流量爆炸
- LatentMoE：shared 专家走全宽，routed 专家在压缩 latent 空间里算
 - K3：shared=2，routed=896 选 16，稀疏度 ≈ 56
- 极端稀疏放大两个失效：4 连乘病态 → 激活爆炸；近千专家 → 均衡失控
- 三招治：Normalized (加 RMSNorm) · SiTU-GLU (有界) · Quantile Balancing

要点：只有把 routed 专家压到 latent 空间，896 选 16 的通信 / 显存才付得起。

SiTU-GLU：有界，又保留 SwiGLU 的形状

为 MXFP4/8 低精度铺路

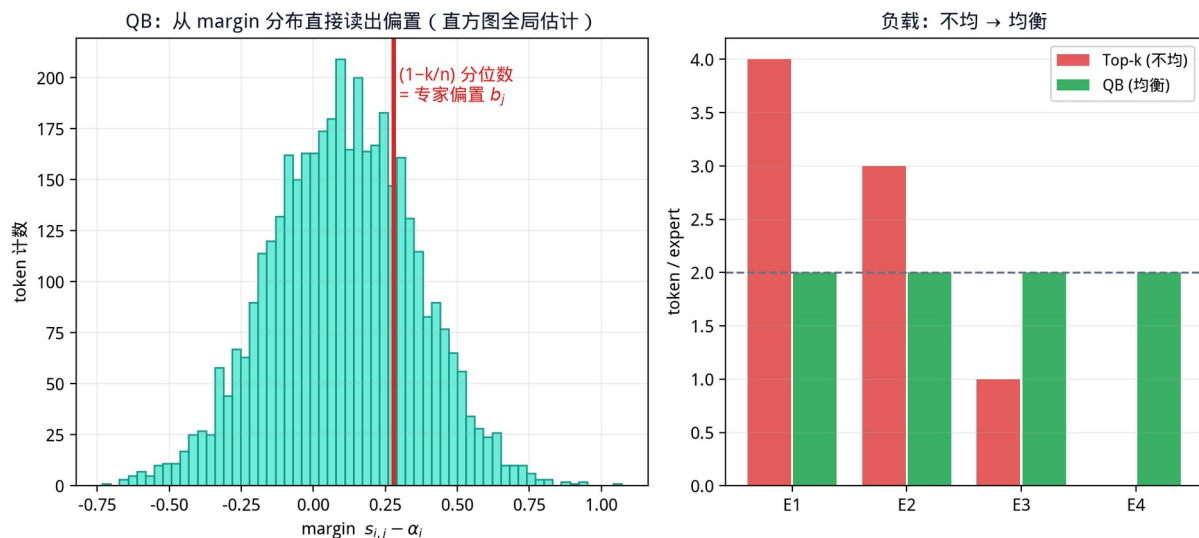
- 背景：SwiGLU 的两个乘子都无界 $\rightarrow$ 大坐标产生激活离群点、低精度易溢出
- SiTU-GLU：对 Swish 线性因子和 up 分支各套一个平滑 cap ($\beta \cdot \tanh$)
 - 近原点一阶匹配 SwiGLU (保留局部行为)
 - $\beta_1, \beta_2 \rightarrow \infty$ 逐点恢复 SwiGLU；输出有界 $\leq \beta_1 \cdot \beta_2 = 100$
- vs 硬 clamp：平滑 cap 在非饱和区保留非零梯度 $\rightarrow$ 训练更好



要点：有界激活是 MXFP4 权重 / MXFP8 激活能训得动的前提之一。

Quantile Balancing : 从『符号步长』到『分位数』

负载均衡升级为有理论最优解



- 背景: aux-loss-free 路由给分数加专家偏置 b_j , 但 b_j 不进 softmax 权重
 - DeepSeek-V3: 固定步长符号更新 → 步长大则震荡、小则慢; 896 专家更难稳
- QB: 把均衡看作最优平衡指派 → 偏置的闭式解 = margin 分布的分位数
 - 无 learning-rate 类超参、几步就均衡
- 工程: 直方图全局估计——一次 all-reduce 求 bin 计数, 与分片方式无关

要点: 把启发式步长升级成『有最优解的分位数』+ 可扩展的直方图估计。

Per-Head Muon + 原生多模态

两个『稳』字诀

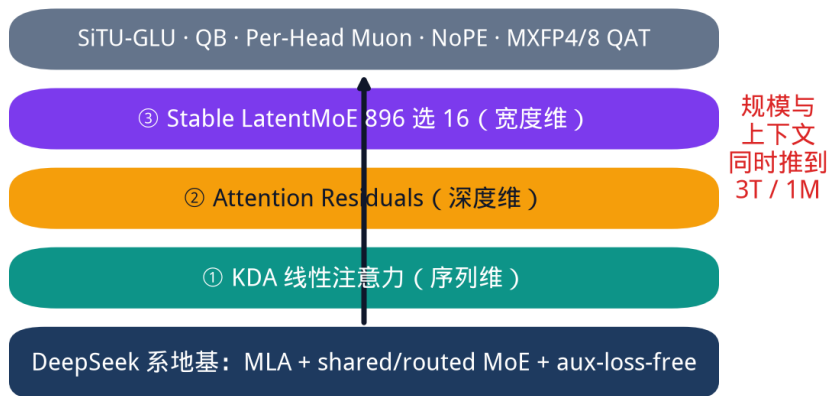
- **Per-Head Muon** : Q/K/V 投影按 head 切块、逐 head 做正交化
 - 整矩阵正交化会让大尺度 head 主导共享更新方向 → 逐 head 均衡各 head 更新
 - 顺带：高瘦 per-head 块的 Newton-Schulz 更便宜
- **MoonViT-V2 (401M) 从零训练，不再用 SigLIP 初始化**
 - 理由是稳定性：SigLIP-init 梯度范数持续偏高 + 尖峰；from-scratch 全程稳
 - 性能打平 → 大规模多模态 LLM 不必依赖对比预训练做初始化

要点：优化器和视觉编码器的改动，共同主题都是『大规模下的稳定性』。

与 DeepSeek-V3 / Qwen3 的对比

K3 = DeepSeek 地基 + 三层新机制

K3 = DeepSeek 系地基 + 三层新机制 + 稳定化/优化器/量化



- 注意力: DeepSeek 全 MLA · Qwen GQA · K3 KDA+MLA 混合
- MoE: DeepSeek 256 选 8 · Qwen 128 选 8(无 shared) · K3 896 选 16 + latent 压缩
- 均衡: DeepSeek 符号步长 · Qwen global-batch aux-loss · K3 Quantile Balancing
- 位置编码: 都用 RoPE · K3 用 NoPE; 优化器: 都用 AdamW · K3 用 Per-Head Muon
- 优势: 长上下文省 · scaling 效率高 · 低精度稳; 代价: 结构 + 系统复杂度显著上升
 - Qwen 走保守稳健路线: 工程简单, 但 scaling/长上下文上限更受限

要点: 激进换来 3T/1M, 代价是复杂度——所以必须配套一整套 Infra。



Part 2

预训练

Scaling Law · 长上下文 · QAT

预训练数据

配方里的两个亮点

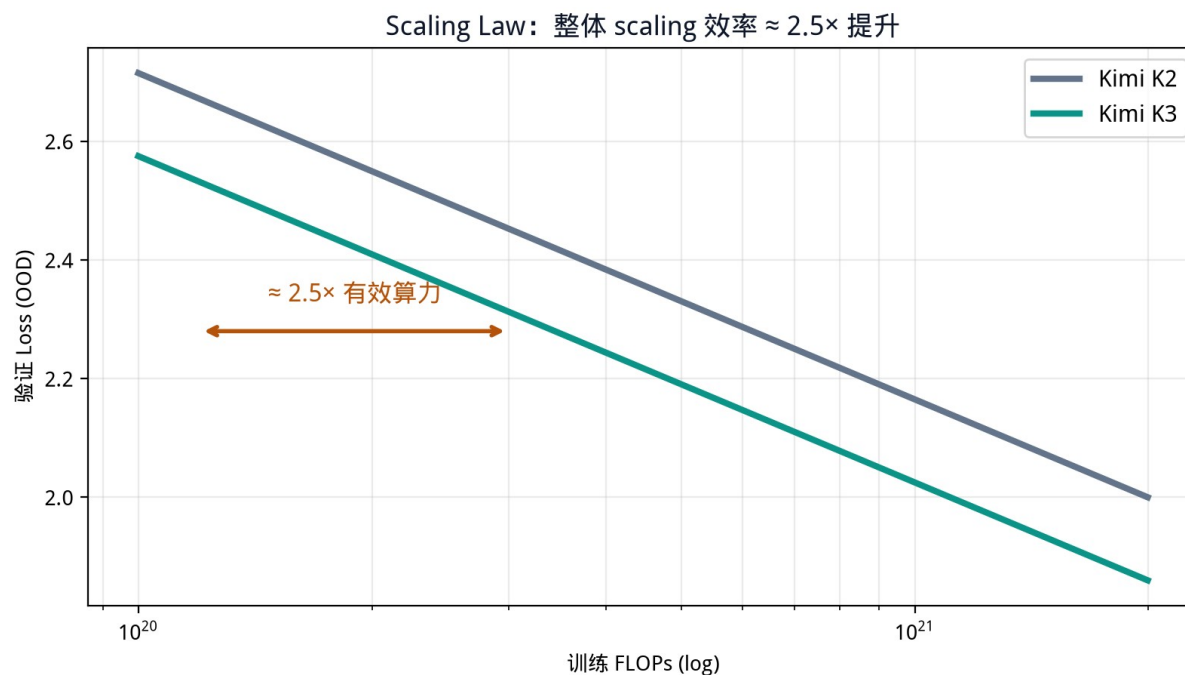
- **四大文本域：Web / Code / Math / Knowledge + 大规模视觉语料**
 - 规则启发式 + 分类器质量打分 + 去重；小模型 ablation 定采样率
- **亮点①：Rephrasing——对 knowledge/math 用多样化 prompt 改写，并对源文档保真校验**
- **亮点②：programmatic 多模态数据——代码片段配其渲染结果**
 - SVG / 3D / 网页 / 游戏 / CAD → 支撑『写代码看效果』的 vision-in-the-loop

要点： programmatic 数据（代码↔渲染）是 K3 视觉编程能力的来源之一。

Scaling Law : 2.5× 与一条方法论

比较训练技巧的正确姿势

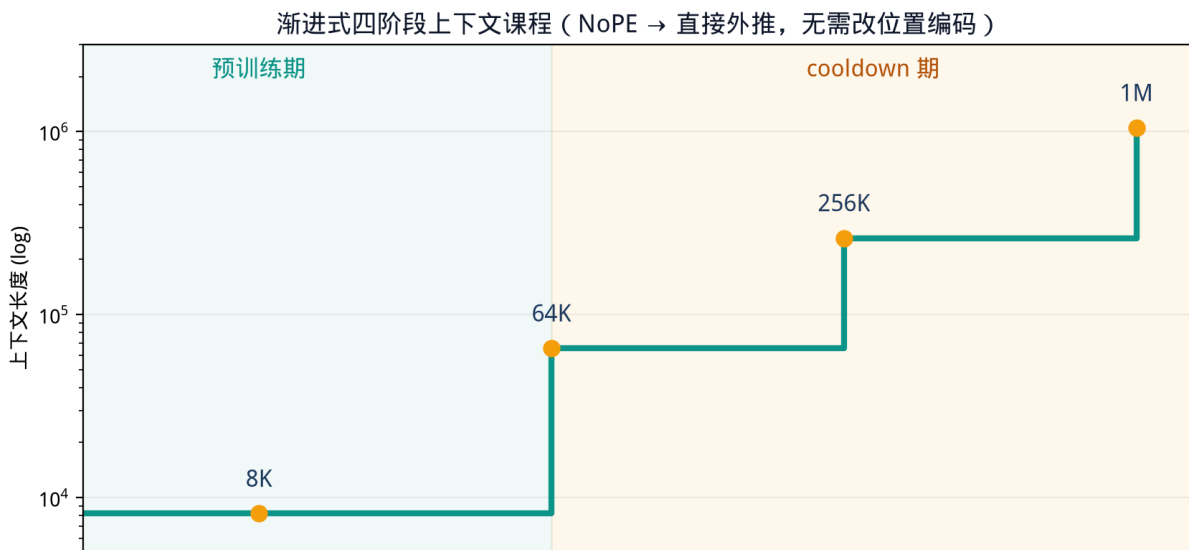
- 结构 / 数据 / 训练一起改 → 最优训练 regime 也变 → 重调 batch/lr/TPP/ 模型 shape
- 结论：整体 scaling 效率 $\approx 2.5\times$ (held-out OOD 验证)
- cosine decay 一致优于 WSD—— 但关键在方法论
 - 两种调度的最优超参差别很大，用同一套超参比较会不公平地偏袒一方
 - 正确做法：各自独立做 scaling-law 搜索，在各自最优下再比



要点：比较任何训练技巧，都要『各自调到最优再比』——否则结论不可信。

长上下文扩展到 1M

NoPE 外推 + 四阶段课程 + 数据合成



- 位置: NoPE → 直接外推 1M, 无需任何位置编码改动
- 数据: 长文 / 长视频含大量低质 → 专门清洗 (去重 / 帧感知哈希 / 结构校验) + 上采样
- 关键: 长度 ≠ 长程能力
 - 合成『必须跨全 1M 才能解』的任务, 把注意力训练在目标尺度, 防退化成局部模式
- 四阶段课程: 预训练 8K→64K, cooldown 256K→1M
 - 把昂贵的长序列计算集中在训练预算的一小部分 → 经济且渐进

要点: NoPE 负责『能外推』, 数据合成负责『真的会用长上下文』。

部署感知量化 (QAT)

训练即量化, 消除 train-infer 失配

- **MoE 专家权重 (参数显存大头) → MXFP4 , 激活 → MXFP8**
 - 非专家部分 (注意力 /latent 投影 /shared 专家 /router) 保持高精度
- **从 SFT 起全程 QAT (覆盖 SFT + RL) → 模型适应量化精度损失**
- **RL 时 rollout 与训练用同一量化方案 → 消除 train-infer 失配**
- **能训得动的底气: SiTU-GLU 有界 + 各处 RMSNorm + KDA 门下界**

MXFP4/8 QAT: 从 SFT 起全程量化感知训练

MoE 专家权重
(参数显存大头)

→ MXFP4 权重 / MXFP8 激活

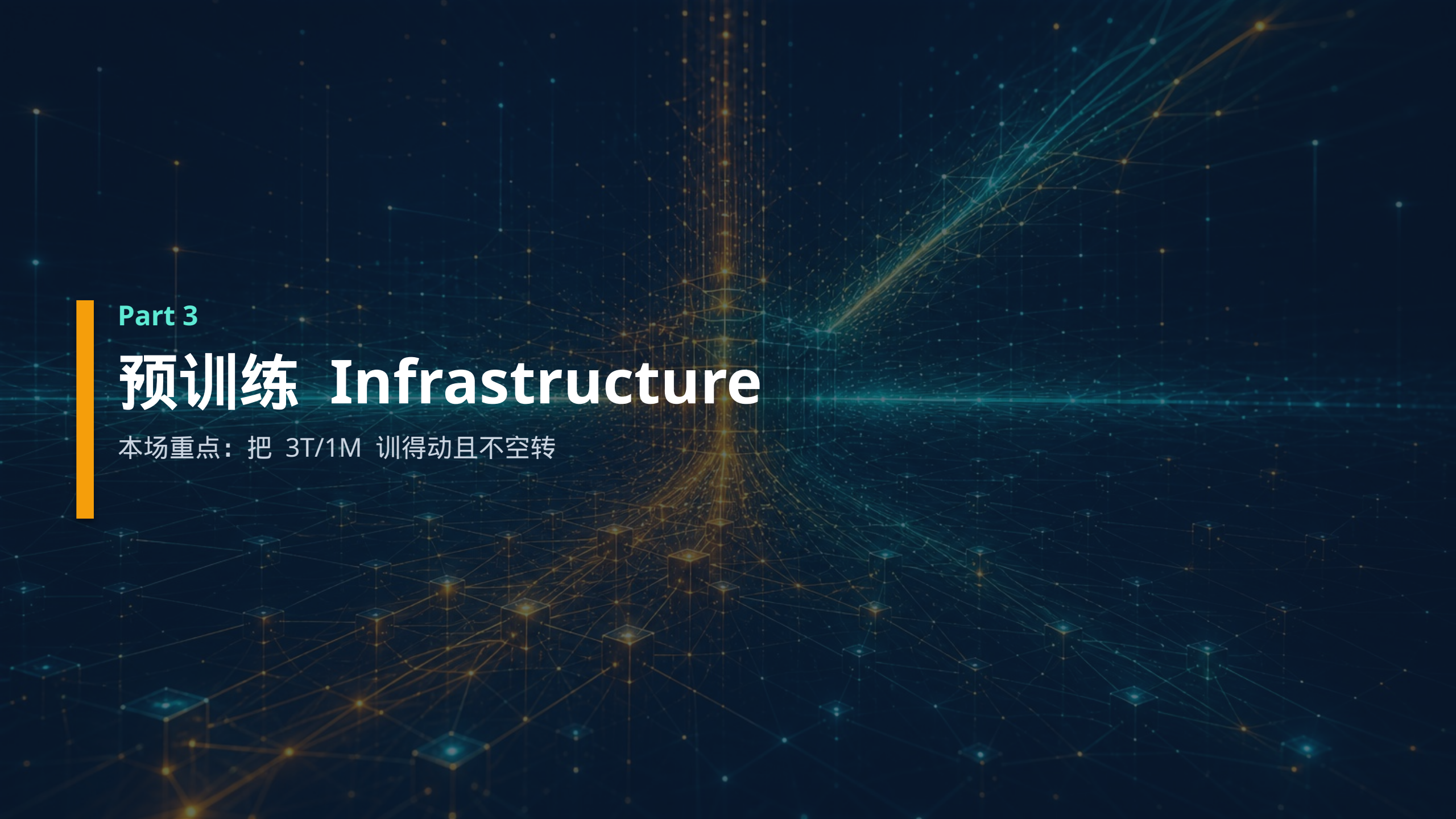
注意力投影 · latent 投影
shared 专家 · router

→ 保持高精度

RL 时 rollout 与训练用同一量化方案 → 消除 train-infer 失配

SiTU-GLU 有界 + 各处 RMSNorm + KDA 门下界 = 低精度的稳定器

要点: 前面那些『有界化』手段, 在这里一起兑现成了 MXFP4 可训。



Part 3

预训练 Infrastructure

本场重点：把 3T/1M 训得动且不空转

为什么 2.8T 预训练首先是『系统问题』

三个数量级事实定义了问题边界

- **参数显存：** 2.78T 参数即使 BF16 也要约 5.5TB → MXFP4 是压显存的前提
- **激活显存：** 93 层 × 1M token × hidden 7168 → TB 级，单一手段不够
- **专家通信：** 896 选 16 的 all-to-all dispatch/combine 是 EP 主带宽消耗
- **三大痛点由此而来**
 - ① EP 负载不均：token 路由数据相关、天然长尾
 - ② 显存超预算：参数 + 激活 + 梯度 + 优化器状态叠加
 - ③ ViT 计算方差：大图 / 长视频让 encoder 计算量剧烈波动，落在关键路径

要点：核心哲学：每个算法选择都要先回答它对『通信 / 显存 / kernel 形状』的影响。

3T 级预训练并行组合

五维并行，各司其职又相互约束

3T 级预训练并行组合

PP (interleaved 1F1B + VPP)

93 层按 stage 切 · VPP 压 bubble

EP (Expert Parallelism)

896 专家按 rank 切 · all-to-all dispatch/combine

ZeRO-1 DP

优化器状态分片 · 扩批量

Pipeline ZeRO-2 梯度分片

梯度按 DP 分片 + 下沉 CPU

CP (KDA Context Parallelism)

1M 序列切分 · KDA 递归态用 KCP

关键约束：EP 的 all-to-all / PP 的 P2P / DP 的 reduce 争同一带宽 → 用计算盖住每一条通信

- PP (interleaved 1F1B + VPP) : 93 层按 stage 切, VPP 压 bubble

- EP : 896 专家按 rank 切, all-to-all dispatch/combine

- ZeRO-1 DP : 优化器状态分片; Pipeline ZeRO-2 : 梯度分片 + 下沉 CPU

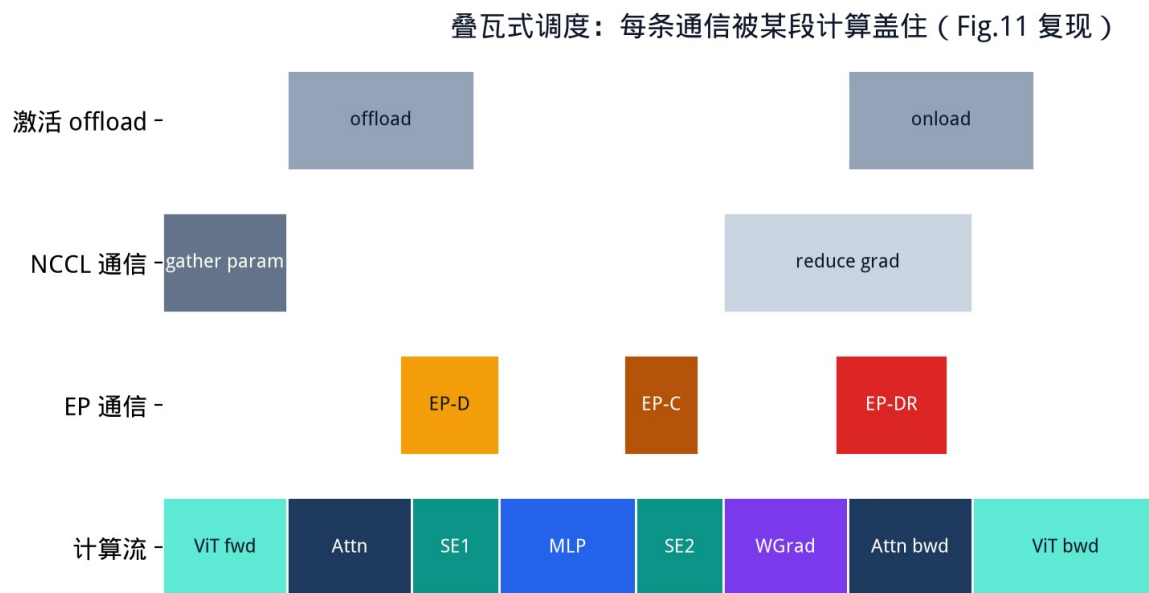
- CP (KCP) : 1M 序列切分, KDA 递归态专门处理
- 关键约束: EP all-to-all / PP P2P / DP reduce 争同一带宽

- → 全篇主线: 用计算盖住每一条通信 (overlap)

要点：这套组合不是『切得多』，而是切完后每条通信都能被某段计算盖住。

叠瓦式调度：每条通信被计算盖住

Fig.11 调度复现

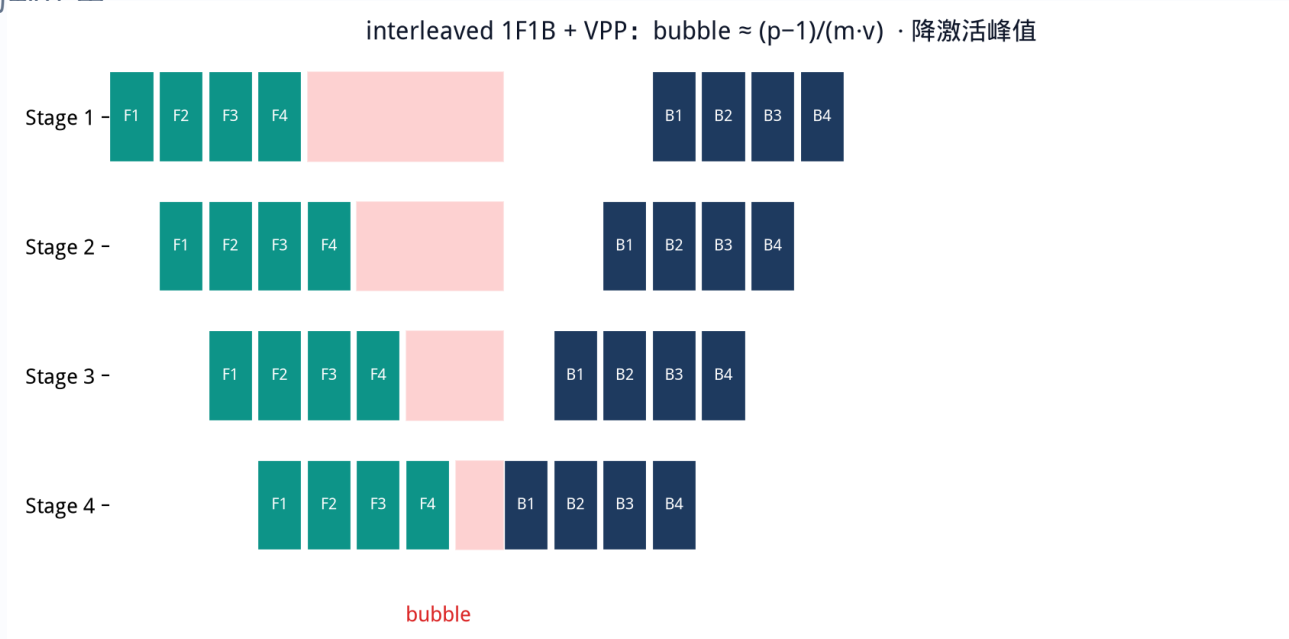


- 四条流并置：计算 / EP 通信 / NCCL 通信 / 激活 offload
- shared expert 拆成 SE1/SE2 派独立 stream → 填 all-to-all 空档
- reduce grad = reduce_scatter + onload + add + offload (Pipeline ZeRO-2 + CPU 梯度)
- EP-DR (dispatch 重算) 在反向 → 对应 memory-efficient MoE

要点：把通信藏进计算的缝隙里，是所有后续优化能成立的基础。

interleaved 1F1B + VPP

压 bubble 的同时留下一个要治的副作用

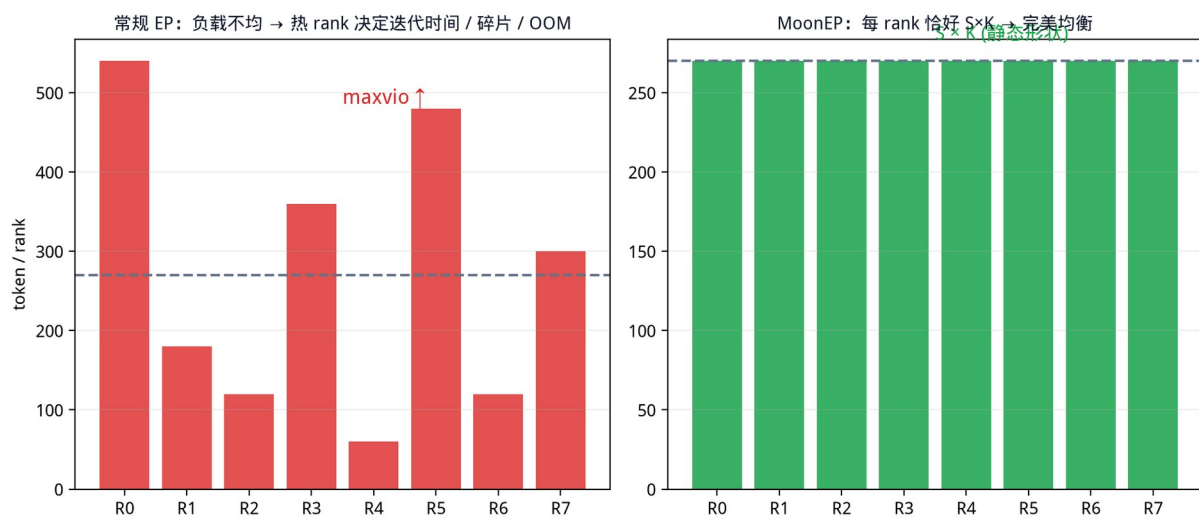


- GPipe bubble 占比 $\approx (p-1)/m$ (p =stage 数, m =micro-batch 数)
- 1F1B : 不改 bubble 比例, 但每 stage 只驻留约 p 份在飞激活 $\rightarrow$ 降峰值
- VPP : 每物理 stage 再拆 v 个虚拟块 $\rightarrow \text{bubble} \approx (p-1)/(m \cdot v)$, 代价是通信 $\times v$
- 副作用: 暖机期各 stage 驻留激活不均 $\rightarrow$ 后面用『PP 激活再平衡』治

要点: VPP 把 bubble 再除以 v , 代价是暖机期激活不均 (后面会治)。

MoonEP：把 EP 负载均衡从『缓解』做成『消灭』

常规 EP 的三连击，其实同源



- 常规 EP 三连击

- ① 热 rank 决定迭代时间 (EP 是同步屏障, 最慢 rank 拖住全部)
- ② 动态形状 → 显存碎片 → 高不均时 OOM
- ③ 每层 host ↔ device 同步: 读回 cu_seqlens 才能按形状发射 grouped GEMM

- 三点同源: 都是『token → 专家映射动态且不均』的后果

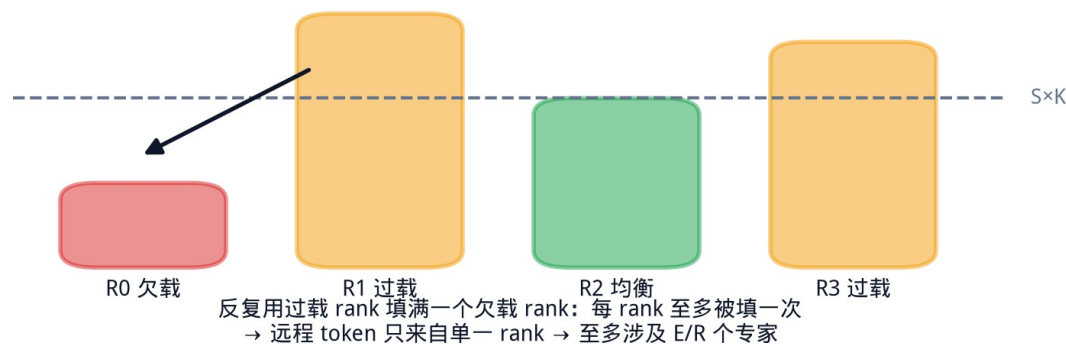
- 治本 = 让映射结果对系统而言变成静态且均衡

要点: 不去『缓解』不均, 而是直接『消灭』它——后面所有红利都从这一步来。

MoonEP 的地基：E/R 冗余专家上界（可证）

这是它区别于 ECHO/UltraEP 的根本

定理：每 rank $\leq E/R$ 个冗余专家的均衡方案必然存在（界基本紧）



预留 E/R 冗余槽 → 规划永远可行、训练永不中断（对比 ECHO/UltraEP 会停训）

- 手段：动态冗余专家——把热专家临时复制到别的 rank，让 token 可拆分摊平
- 定理：任意路由下，每 rank $\leq E/R$ 个冗余专家的均衡方案必然存在（界基本紧）
 - 证明构造：反复用过载 rank 填满一个欠载 rank；每 rank 至多被填一次
 - → 远程 token 只来自单一 rank → 至多涉及 E/R 个专家
- 含义：预留 E/R 冗余槽 → 规划永远可行、训练永不中断
 - 对比 ECHO/UltraEP：固定冗余数 / token cap，无解时被迫停训 + 手调

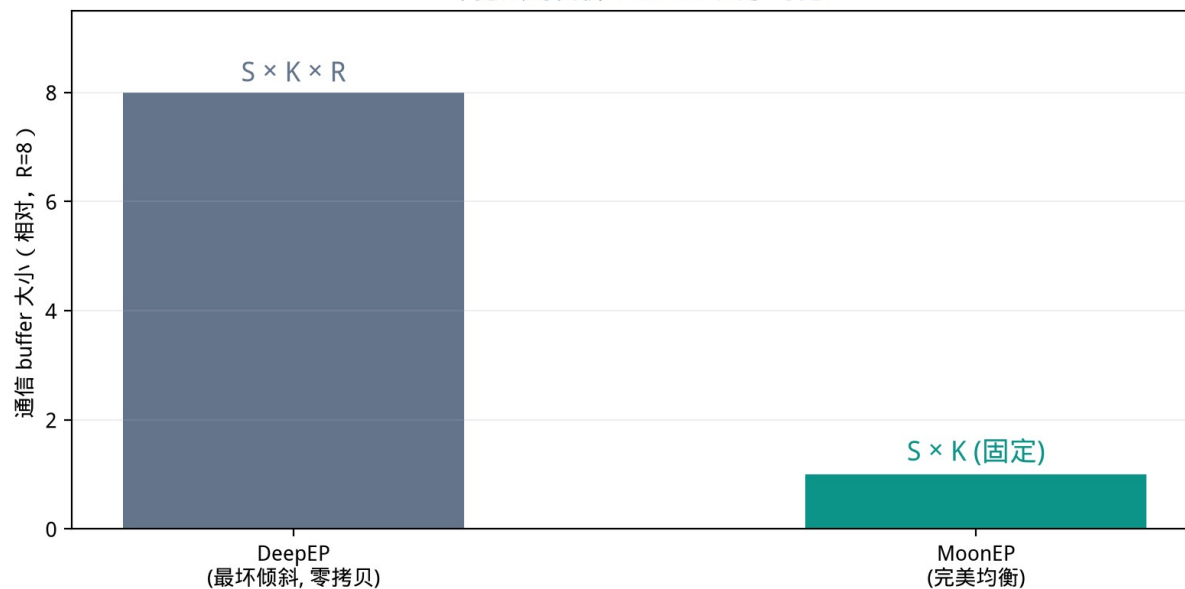
要点：不是『预设一个够用的冗余数』，而是『证明了一个恒成立的上界』。

MoonEP：均衡换来的三个免费红利

零拷贝 · 静态形状 · 无碎片

- **零拷贝**：规划 kernel 预算每 token 目的地 → 直接写到远端最终槽位，buffer 视图交给计算
 - DeepEP 最坏要 $S \times K \times R$ 的 buffer；MoonEP 完美均衡恒定 $S \times K$ ，与倾斜无关
- **静态形状**：每层 MoE 形状静态已知 → 消除 host 同步、降 kernel 启动开销
 - rank 内仍偏斜 → workload-aware GEMM 调度（代价模型 + 离线 autotune）+ 共享专家单独 stream
- **无碎片**：静态形状 × 单内存池 → 几乎不再碎片化（均衡直接反哺显存）

零拷贝的代价：buffer 大小对比

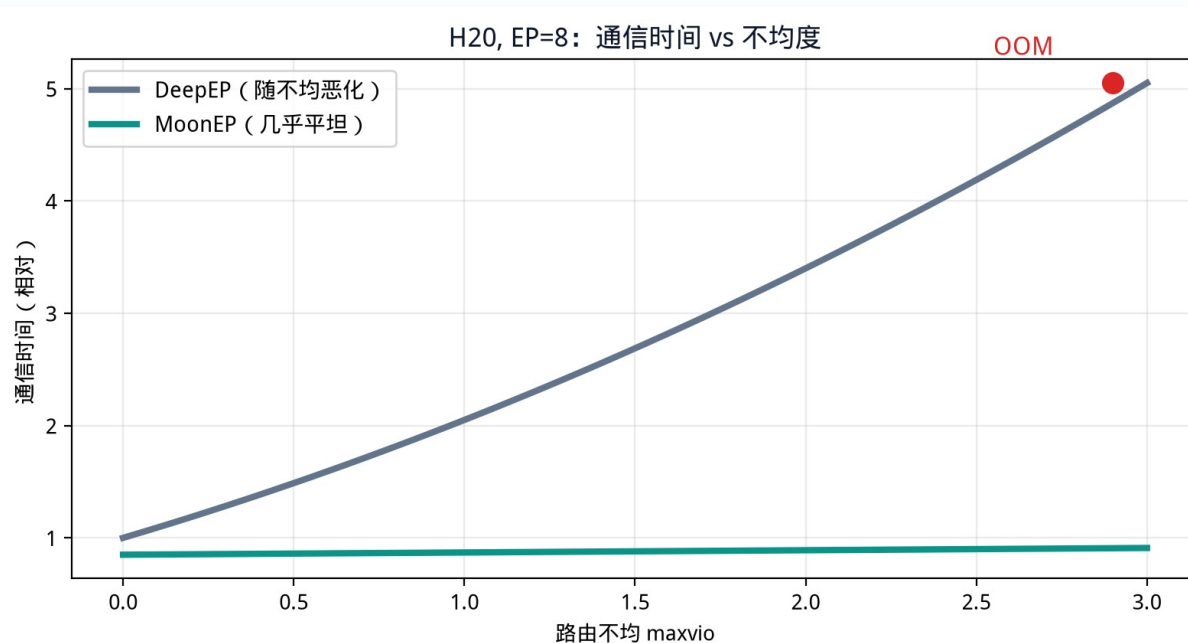


要点：完美均衡是因，零拷贝 / 静态形状 / 无碎片都是果。

MoonEP 基准 (H20, EP=8)

通信时间随不均『几乎平坦』

- MoonEP : 通信时间随 maxvio 几乎不变 (buffer 不膨胀、路径无拷贝)
- DeepEP : 随不均稳步恶化 (延迟由最热 rank 决定) , 高不均直至 OOM
- 端到端: MoonEP 各不均水平迭代时间持平、不 OOM
 - 代价: 冗余专家要预取权重、反向要额外 reduce → 相对停训 /OOM 可忽略

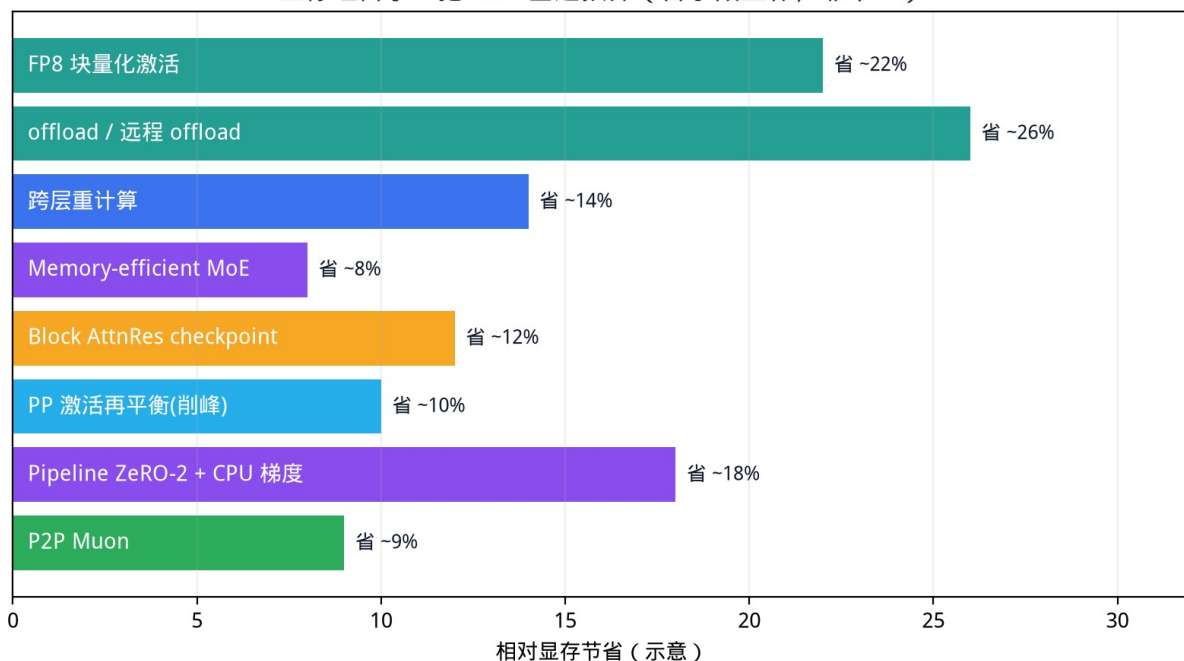


要点: 用很小的额外开销, 换来对负载不均的『免疫』。

显存组合拳：把 2.8T 塞进预算

显存是全局零和，靠组合而非单一手段

显存组合拳：把 2.8T 塞进预算（各手段互补，非单一）



- 统一激活管理器：重计算 / FP8 块量化 / offload / 远程 offload 都是『存储策略』

– 张量粒度自由组合、注解声明、与模型代码解耦；单内存池避免多流碎片

- Memory-efficient MoE：梯度数学变换免存前向输出；反向重算 dispatch 并 overlap

- Block AttnRes：块表示生成一次共享 + 整体 checkpoint → 回到标准残差显存足迹

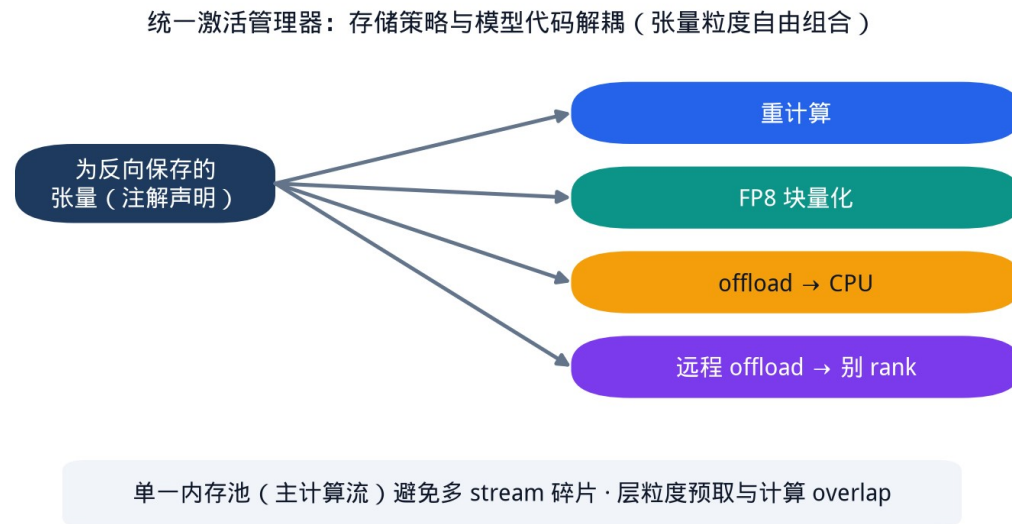
- 省下来的每 GB → 支撑更长序列 / RL 的外部 KV pool

要点：没有银弹——八种手段各省一点，合起来才把 2.8T 塞进预算。

显存组合拳（续）：三个系统级手段

把压力从 HBM 挪到 CPU / 别的卡 / P2P

- 统一激活管理器（图示）：张量 → 可插拔存储后端，与模型代码解耦
- PP 激活再平衡：1F1B 暖机让各 stage 激活不均 → Mooncake 远程 offload 削峰
- Pipeline ZeRO-2 + CPU 梯度：梯度分片下沉 CPU，GPU 只留 double buffer
- P2P Muon：正交化需完整矩阵，改为只 P2P 拉『自己负责更新的』分片
 - 消除全参 all-gather 的显存 + 通信双爆炸，按 model-chunk 流水隐藏



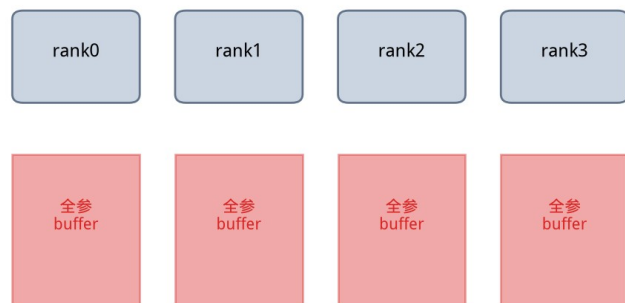
要点：削峰而非降总量——显存瓶颈是『单卡峰值』，不是『总量』。

P2P Muon : 定向通信 > 广播

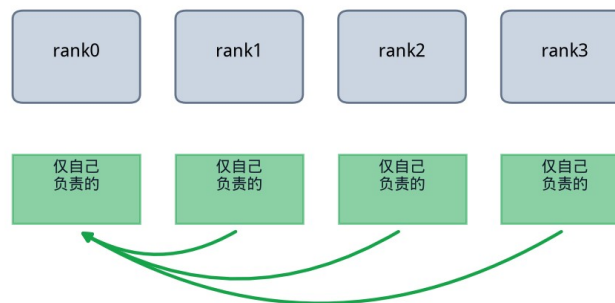
只有『要更新该参数的卡』才需要它的参数碎片

朴素：全量 all-gather

K3：P2P 定向拉分片



显存 $O(P)/rank + all-gather\ O(P)$ 通信



buffer $O(P/DP)$ · 消除全参 buffer · 通信流水隐藏

- 约束：Newton-Schulz 是整矩阵多项式，必须完整矩阵，无法在分片上独立做
- 朴素：全量 all-gather → 每卡多一份全参 buffer（显存）+ all-gather 全参（通信）
- K3：每卡只 P2P 拉回自己负责更新的参数分片（属主 → 更新者定向）
 - 临时 buffer 从 $O(P)$ 降到 $O(P/DP)$ ，消除全参 buffer；通信流水隐藏

要点：想清楚『谁真正需要这份数据』，就能把 all-gather 降级为点对点。

KDA 协同 + 多模态 encoder

KCP 跨设备 · ViT 塞进流水气泡

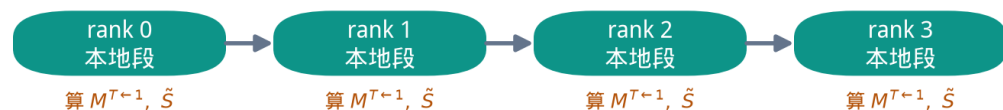
- **KCP：线性注意力跨设备 CP 只传固定大小状态**
(softmax 要传随长度增长的 KV)

- vanilla 『从 $S=0$ 求和』对 KDA 不成立：转移矩阵 M_t 要作用在入态上
- KCP 把每段拆成两个本地可算量 → 前缀扫描恢复入态，一次固定大小 all-gather

- **Bubble filling：把 ViT 前向 / 反向塞进 PP 流水气泡**
(承 K2.5 DEP，思路同 Optimus)

- ViT 可搬动、无跨 micro-batch 依赖 → 有效开销基本被隐藏

KDA Context Parallelism: 转移矩阵分解 + 前缀扫描



一次固定大小 all-gather 交换 (M, S) → 前缀扫描恢复各 rank 入态

线性注意力只传固定大小状态 (softmax 需传随长度增长的 KV 块)

vanilla 『从 $S=0$ 求和』对 KDA 不成立：转移矩阵 M_t 要作用在入态上

要点：思想迁移：把 softmax 的 CP 用『转移矩阵分解 + 前缀扫描』搬到线性注意力。



Part 4

后训练 / RL

简化：三阶段 + 9 专家 + 蒸馏

RL 全景：三阶段 + 9 个专家

SFT → 按域 × effort 做 RL → 蒸馏合并

- **SFT 冷启动** → 三大域各训一个专家 × 三档 effort = 9 个专家 → 合并
 - 三大域：general / general agents / coding agents
- **partial rollout**：λ 比例轨迹完成就暂停生成、让策略优化先走（避免长尾 straggler）
 - 长轨迹跨多迭代 → 数据 staleness → 逐 token 正则容忍极端 off-policy
- **reasoning-effort RL**：给 token 预算，超预算给 -1 → 得到 max/high/low 三档
- **Agentic GRM**：锦标赛式二元比较 + rubric 协议 + verbosity 预算防 reward hacking

要点：不训单任务专用模型，而是『域 × effort』九宫格再蒸馏成一个。

蒸馏合并 + 部署感知后训练

把 9 个专家收敛成一个可部署模型

- **MOPD 多教师 on-policy 蒸馏**：按域 $\times$ effort 用对应教师，逐 token 稠密奖励
 - 稠密信号天然融入 RL 框架、支持 partial rollout
- **Draft / EAGLE-3**：把预训练 MTP 层微调成投机解码 draft
 - 输入融合第 1/4/ 末 AttnRes block 的低 / 中 / 高层特征
- **LK loss**：直接优化投机解码的『接受率』而非 KL 代理
 - RL 环境：统一 white-box harness · 知识图谱任务合成 · kernel/AET/web dev

要点：9 个专家最终蒸馏回一个模型，且 draft 复用了预训练的 MTP 层。

评测速览

紧随最强闭源，稳定领先其他开源

- **整体：**紧随 Claude Fable 5 / GPT-5.6 Sol ， 稳定领先其他开源
- **领先 /SOTA：** ProgramBench · SWE-Marathon · BrowseComp · OmniDocBench
 - WebDev Arena 首个登顶的开源模型
- **短板：** research 级推理 (HLE/CritPt) · 部分知识工作 Elo · hardened exploit
- **成本效率：** 多个 coding/agent 榜处于成本 - 效率前沿
- **Case：** GPU kernel 优化 · 从零写编译器 MiniTriton · 48h 自主设计推理芯片

要点： 开源前沿：能力接近闭源第一梯队，成本效率是亮点。

演进脉络：每一代验证一个新机制

K2 → K2.5 → Kimi Linear → AttnRes → K3

- K2 : 全 MLA · 384 选 8 · Muon · SwiGLU · 128K
- K2.5 : 视觉 agentic · Decoupled Encoder Process
- Kimi Linear : 提出 KDA · 3:1 KDA:MLA · 验证线性注意力可超全注意力
- Attention Residuals : 深度维注意力 · 48B 规模验证
- Kimi K3 : 把 KDA + AttnRes + LatentMoE + Per-Head Muon + QAT + 全套 Infra 集成到 2.8T/1M

要点：K3 是集大成——每一代先单独验证一个机制，再在 K3 集成。

收尾：五条可迁移的方法论

给我们自己项目的启示

- ① 算法 - 系统协同是主线：改算法先想它对 kernel/ 通信 / 显存的影响
 - KDA 加下界 → 对角块上 Tensor Core ; MoonEP 完美均衡 → 静态形状 + 零拷贝 + 免同步
- ② 把成熟维度的思想迁到新维度：attention 迁到深度维；CP 迁到线性注意力
- ③ 有界化 / 归一化 = 极端稀疏 + 低精度下的稳定器（SiTU-GLU/RMSNorm/ 门下界）
- ④ 比较训练技巧要『各自调到最优再比』（cosine vs WSD 的教训）
- ⑤ 负载均衡从『启发式步长』升级为『有最优解的分位数』（QB + 直方图）

要点：最好的系统优化，往往始于一个小的算法改动。

Q & A · 谢谢！

备问：3:1 为何不全 KDA · NoPE 外推 · 896 选 16 如何训得动 · AttnRes 推理开销 · MoonEP 与 DeepEP 差异 · 与 DeepSeek 到底差在哪

详见 [DESIGN_NOTES.md](#) 与 [PRETRAIN_INFRA_DEEPDIVE.md](#)